

本章目标

掌握 C++ 函数相对 C 函数的全部增强:引用参数、默认参数、函数重载、const 成员函数、函数对象、lambda 与 std::function;理解三种传参方式的取舍,能够在工程中写出接口清晰、复用性强的函数。

前置知识

会写 C 函数(返回值、形参、递归);理解指针与取址;了解第1章的引用概念与第3章的循环。本章代码均基于 C++17。

\$4.1 声明与定义分离	\$4.2 传值、传引用、传指针	\$4.3 默认参数与重载
\$4.4 const 成员函数	\$4.5 函数指针与函数对象	\$4.6 lambda 表达式

\$4.1 从 C 函数到 C++ 函数:声明与定义

C 程序员对函数已经很熟悉:返回类型、形参列表、局部变量、递归调用。C++ 的普通函数与 C 几乎完全兼容,但增加了三样 C 没有的东西——引用参数、默认参数、函数重载——并为后续的成员函数与 lambda 提供了基础。先看最基本的一条纪律:声明与定义分离。

声明只告诉编译器函数的存在与签名,定义给出函数体。工程上声明放在头文件,定义放在源文件:多个源文件包含同一个头文件时,各自编译,互不干扰,最后在链接阶段把符号合并。这与 C 的做法一致,但 C++ 的类型检查更严格:重载函数的调用必须与某个签名精确匹配或可转换匹配,否则直接编译失败,而不是 C 那种"名字对上就链接"的宽松模型。

这里有一个 C 程序员必须理解的概念:名字修饰。C++ 编译器会把函数名与参数类型编码成底层符号,例如 `int f(double)` 会被修饰为内部符号,两个参数不同的同名函数因此可以并存——这就是重载的底层机制。代价是 C 语言不认这些符号,所以调用 C 库或让 C 代码链接 C++ 代码时,要用 `extern "C"` 关闭修饰。这是多语言项目中的高频知识点,第13章讲模块系统时会再见到它。

清单 4-1 声明与定义分离,以及 `extern "C"`

```
// 声明:只给签名,放头文件 math_util.h
int add(int a, int b);
double avg(const double* data, int n);
extern "C" int c_version(void); // 关闭名字修饰,供 C 代码调用

// 定义:给出函数体,放源文件 math_util.cpp
#include "math_util.h"
int add(int a, int b) { return a + b; }

double avg(const double* data, int n) {
    double sum = 0.0;
    for (int i = 0; i < n; ++i) sum += data[i];
    return n > 0 ? sum / n : 0.0;
}

int c_version(void) { return 42; } // 被 C 程序以普通函数名调用
```

\$4.2 传值、传引用、传指针

C 里向函数传数据只有两条路:传值(拷贝)或传指针(传地址)。C++ 增加了一条路:传引用。引用是变量的别名,声明时必须绑定对象,永远不为空;对引用的修改就是对实参的修改。底层实现与指针一样都是地址,但语法上直接写变量名,并且编译器保证它非空,不需要判空分支。

三种方式的适用场景可以总结成一句话:小对象传值,大对象传 `const` 引用,要修改实参就传引用,允许为空(表示"无")就传指针。所谓大对象,指 `std::string`、`std::vector` 这类内部持有一块堆内存的类型——按值传递会触发一次拷贝,拷贝一个百万元素的 `vector` 就是全量复制,代价高昂;传 `const` 引用只传递一个地址,零拷贝且保证不被修改。

C 的写法	C++ 的写法	说明
void f(int x)	void f(int x)	传值,修改形参不影响实参,两者相同
void f(int* p)	void f(int* p)	传指针,可为空,调用方需判空;两者相同
无	void f(int& x)	传引用,别名语义,非空保证,修改即修改实参
void f(const int* p)	void f(const int& x)	只读传参,C 用 const 指针,C++ 首选 const 引用

还有一个 C 程序员容易忽略的细节:const 引用可以绑定临时对象,并且会把临时对象的生命周期延长到引用本身失效为止;普通引用不能绑定临时对象。这保证了 `avg({1.0, 2.0, 3.0}, 3)` 这类传临时数组的调用是安全的。反过来,把临时对象绑定给普通引用、或者让函数返回局部对象的引用或指针,都会制造悬空引用——对象在函数返回后已经销毁,访问它是未定义行为。编译器通常只给出警告,不会拒绝编译,这是 C 系语言最经典的一类 bug。

清单 4-2 三种传参方式对比

```
#include <iostream>
#include <string>

void byValue(int x) { x = 100; }           // 修改形参,不影响实参
void byRef(int& x) { x = 100; }           // 修改引用,即修改实参
void byPtr(int* p) { if (p) *p = 100; }   // 指针可为空,务必判空

double total(const std::vector<double>& v) { // const 引用:只读零拷贝
    double s = 0.0;
    for (double x : v) s += x;
    return s;
}

int main() {
    int a = 1, b = 1, c = 1;
    byValue(a);                          // a 仍为 1
    byRef(b);                             // b 变为 100
    byPtr(&c);                             // c 变为 100
    byPtr(nullptr);                       // 安全:内部判空
    std::cout << a << " " << b << " " << c << "\n"; // 1 100 100
    return 0;
}
```

⚠ 易错点 1:返回局部对象的引用或指针

函数返回局部变量时,对象在函数退出时销毁,引用与指针随即悬空。正确做法是按值返回:现代编译器普遍实现了返回值优化(RVO)和移动语义,大对象按值返回不会产生多余拷贝,放心使用。

§4.3 默认参数与函数重载

默认参数让调用方可以省略部分实参。两条硬规则:第一,一旦某个参数有默认值,它右边的所有参数都必须有默认值,这就是靠右原则——否则省略实参时无法确定省略到哪一位;第二,默认值写在声明处(头文件),定义处不能再写,否则编译器报“重复默认实参”错误。头文件的读者一眼就能看到默认值,而每个源文件只看到自己 include 的声明,行为一致。

函数重载允许同名函数携带不同的参数列表,调用时编译器根据实参的类型、个数挑选最合适的版本。注意两点:重载只看参数,不按返回值区分——只改返回类型、参数完全相同的两个函数无法重载,因为调用语句无法据此选择;参数个数相

同但类型可转换时,编译器按优先级挑选:精确匹配优于整型提升(如 int 升 long),提升优于标准转换(如 double 转 int),转换优于用户定义转换,转换代价相同的调用直接判为二义性错误。

C 的写法	C++ 的写法	说明
print_int(5); print_str("hi");	print(5); print("hi");	C 靠不同名字表达不同输入,C++ 同名重载
int clamp(int v, int lo, int hi)	int clamp(int v, int lo, int hi)	C 只能给三个参数,C++ 可给默认值省去调用
编译期无法检查	参数不匹配直接编译失败	C++ 强类型签名,错误更早暴露

清单 4-3 默认参数与重载

```
#include <iostream>
#include <string>

// 默认参数:靠右,写在声明处
void logMessage(const std::string& msg, int level = 0);

// 重载:按参数类型区分
void print(int v) { std::cout << "整数:" << v << "\n"; }
void print(double v) { std::cout << "浮点:" << v << "\n"; }
void print(const std::string& s) { std::cout << "文本:" << s << "\n"; }

void logMessage(const std::string& msg, int level) { // 定义处不写默认值
    std::cout << "[L" << level << "] " << msg << "\n";
}

int main() {
    logMessage("启动");           // 省略 level,取默认值 0
    logMessage("警告", 2);        // 显式传 level
    print(5);                     // 精确匹配 print(int)
    print(3.14);                 // 精确匹配 print(double)
    print(std::string("hi"));     // 精确匹配 print(string)
    return 0;
}
```

 **易错点 2:默认参数与重载叠加产生二义性**

void f(int a, int b = 1); 与 void f(int a); 同时存在时,调用 f(5) 既可以解释为"第一个函数用默认参数",也可以解释为"精确调用第二个",编译器无法抉择,直接报错。设计接口时,默认参数与重载不要表达同一组调用,二选一即可。

§4.4 成员函数与 const 成员函数

C 没有"对象调用函数"的语法,只能写 struct 加全局函数,把对象指针显式传进去。C++ 把函数放进类里成为成员函数,对象用点号调用,编译器隐式地把对象的地址作为 this 指针传给函数。this 是 const 指针:指针本身不能改指向,但指向的对象可以修改。

const 成员函数是在参数列表之后加 const 修饰的函数,例如 std::string getName() const。它的语义是:this 变为指向 const 对象的指针,函数体内不能修改任何非 mutable 成员。规则反过来更常用:const 对象只能调用 const 成员函数;非 const 对象可以调用任意成员函数。于是"这个函数不改对象"成为接口契约的一部分,编译器强制检查——这在 C 里没有对应物,是 C++ 用类型系统防呆的典型例子。

偶尔需要在 const 成员函数里修改某个成员,比如统计调用次数、缓存计算结果,可以把该成员声明为 mutable。mutable 意味着"即使对象是 const,这个字段也可以改",它只应出现在内部实现细节上,不该出现在公开接口数据中。

C 的写法	C++ 的写法	说明
void set_name(Student* s, const char* n)	s.setName(n)	C 显式传对象指针,C++ this 隐式传递
无	void get() const;	const 成员函数,编译器检查只读性
const Student* s; s->set()	const Student s; s.set()	const 对象禁止调用非 const 成员,C++ 编译期拒绝

清单 4-4 类、this 与 const 成员函数

```
#include <iostream>
#include <string>

class Student {
public:
    Student(std::string n, double s) : name(n), score(s) {}

    const std::string& getName() const { return name; } // const 成员函数
    double getScore() const { return score; }          // 只读,任何对象可调

    void rename(const std::string& n) { name = n; }     // 非 const,修改对象
    void recordCall() const { ++calls; }               // mutable 计数

    int callCount() const { return calls; }

private:
    std::string name;
    double score;
    mutable int calls = 0; // const 成员函数里可以修改
};

int main() {
    Student s("张三", 85.5);
    s.rename("张三丰");
    const Student& c = s; // const 引用视角
    std::cout << c.getName() << " " << c.getScore() << "\n";
    // c.rename("李四"); // 错误:const 对象不能调非 const 成员函数
    c.recordCall();
    std::cout << "调用次数:" << s.callCount() << "\n";
    return 0;
}
```

§4.5 函数指针、函数对象与 std::function

函数指针与 C 完全一致:它保存函数的地址,通过它调用函数。C 里回调机制完全依赖函数指针,还要额外传一个 void* 上下文才能携带状态。C++ 在函数指针之上提供了两种更强大的可调用对象。

函数对象是重载了 operator() 的类,对象用起来像函数,但可以携带成员变量作为状态。C 的静态函数天然无状态,要用全局变量模拟,而函数对象的每个实例都有独立状态,天然线程安全。lambda 是函数对象的语法糖(见 §4.6),手写函数对象的场景已经很少,但理解它才能理解 lambda 的本质。

std::function 是通用的可调用包装器,来自 <functional> 头文件:函数指针、函数对象、lambda 都可以赋值给它,调用时统一用 f(args) 语法。它内部做类型擦除,把不同的可调用类型收进同一个类型,适合做回调参数、事件表、策略注入。代价是一次间接调用,比裸函数指针略慢,回调非热点路径时通常可忽略。

C 的写法	C++ 的写法	说明
void apply(int* d, int n, int (*f)(int))	void apply(vector<int>&, function<int(int)> f)	C 只认函数指针,C++ 接受任意可调用对象
void* ctx; f(ctx, x)	[&ctx](int x){ ... }	C 用 void* 传上下文,C++ 用 lambda 捕获
回调状态靠全局变量	函数对象成员变量即状态	C++ 状态与调用者绑定,可并发安全

清单 4-5 三种可调用对象并存

```
#include <iostream>
#include <functional>

int triple(int x) { return x * 3; } // 普通函数

struct Multiplier { // 函数对象:携带状态
    int factor;
    int operator()(int x) const { return x * factor; }
};

void run(const std::function<int(int)>& f, int v) { // 统一回调接口
    std::cout << "结果:" << f(v) << "\n";
}

int main() {
    run(triple, 10); // 函数指针
    run(Multiplier{7}, 10); // 函数对象
    run([](int x) { return x * x; }, 10); // lambda
    // 三者在同一个 std::function 参数上平等互换
    return 0;
}
```

§4.6 lambda 表达式

lambda 的完整语法是 [捕获列表](参数列表) -> 返回类型 { 函数体 },C++14 起返回类型可以省略让编译器推导,C++17 起支持泛型 lambda:[](auto x) 参数类型由实参推导,相当于模板函数。捕获列表决定 lambda 能访问外部哪些变量:[] 不捕获; [=] 按值捕获所有用到的自动变量; [&] 按引用捕获; [x, &y] 指定捕获,按值捕获 x、按引用捕获 y; [this] 捕获当前对象的指针。

理解 lambda 的关键是知道它的本质:编译器为每个 lambda 生成一个匿名类,捕获列表就是类的成员变量,函数体就是 operator()。也就是说 lambda 不过是我们手写函数对象的一份语法糖——按值捕获是拷贝成员,按引用捕获是引用成员,生命周期规则完全一致。因此“按引用捕获局部变量,却把 lambda 存到比局部变量活得更久的地方”就会产生悬空引用,这是 lambda 最常见的坑。

工程上,lambda 最常出现在标准库算法里:排序比较器、查找谓词、转换函数都是一行 lambda 的事。写函数对象和函数指针的机会因此大幅减少,接口参数尽量用 lambda 或 std::function 接收。

清单 4-6 lambda 的捕获与泛型用法

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> v{5, 3, 8, 1, 9, 2};

    // [&] 按引用捕获:排序比较器
    std::sort(v.begin(), v.end(), [](int a, int b) { return a < b; });

    int threshold = 4;
    // [=] 按值捕获:统计大于阈值的个数
    int cnt = (int)std::count_if(v.begin(), v.end(),
                                [=](int x) { return x > threshold; });

    // 泛型 lambda:任何可比较类型都能用
    auto maxOf = [](auto a, auto b) { return a > b ? a : b; };

    // 指定捕获:混合按值与按引用
    int base = 100;
    auto shift = [base, &cnt](int x) { return x + base + cnt; };

    std::cout << "大于" << threshold << "的个数:" << cnt << "\n";
    std::cout << "max:" << maxOf(3, 7) << ", shift:" << shift(1) << "\n";
    return 0;
}
```

⚠ 易错点 3:按引用捕获后逃逸作用域

`auto f = [&]{ return x; };` 把 `f` 存到全局或容器里,等 `x` 所在函数返回后再调用 `f`,引用的对象已经销毁。排查口诀:按值捕获是安全的拷贝,按引用捕获的 `lambda` 只能在使用它的同一作用域内存活。

★ 关键点:传参选择速记

只读小对象传值;只读大对象传 `const` 引用;要修改传引用;允许为空传指针;返回值按值返回,依赖 RVO 与移动语义。这条经验能避开九成参数设计错误。

函数相关速查表

概念	语法示例	要点
函数声明	<code>int add(int, int);</code>	只给签名,放头文件
函数定义	<code>int add(int a, int b) { ... }</code>	给出函数体
传值	<code>void f(int x)</code>	拷贝,修改不影响实参
传引用	<code>void f(int& x)</code>	别名,非空,修改即实参
传 const 引用	<code>void f(const int& x)</code>	只读零拷贝,可绑临时对象
传指针	<code>void f(int* p)</code>	可为空,函数内判空
默认参数	<code>void f(int a, int b = 1)</code>	靠右原则,声明处给出
函数重载	<code>void f(int); void f(double);</code>	按参数区分,不按返回值
extern "C"	<code>extern "C" int cf(void);</code>	关闭名字修饰,兼容 C
函数指针	<code>int (*p)(int) = triple;</code>	与 C 相同,存函数地址
函数对象	<code>struct F { void operator()() {} };</code>	重载调用运算符,携带状态
std::function	<code>std::function<int(int)> f = g;</code>	类型擦除,统一可调用对象
const 成员函数	<code>void get() const;</code>	this 只读,mutable 例外
lambda	<code>auto f = [=](int x) { ... };</code>	闭包语法糖,捕获列表即成员

自测题

- Q 1. 传值、传引用、传指针三者的本质区别是什么?什么时候必须用指针?**
- Q 2. 为什么默认参数必须靠右?声明与定义分离时默认值写在哪儿?**
- Q 3. 为什么重载不能只靠返回值区分?重载决议的优先级顺序是什么?**
- Q 4. const 成员函数里能修改成员变量吗?mutable 解决什么问题?**
- Q 5. lambda 的 [=] 与 [&] 捕获有何区别?按引用捕获的 lambda 逃逸出作用域会怎样?**

✅ 参考答案

- 传值复制一份实参,修改形参不影响实参;传引用与传指针都把地址传进去,区别是引用非空且语法自然,指针可为空。需要表达"可能没有值"(如空指针表示空),就必须用指针;其余修改实参的场景优先引用。
- 调用方省略实参时,编译器必须知道省略到哪一位,靠右原则保证省略的总是末尾一段连续参数;默认值写在声明处(头文件),定义处再写属于重复默认实参,编译报错。
- 调用语句 `f(x)` 只提供实参,无法根据"将来的返回值类型"做选择,所以仅返回值不同的重载必然二义。决议优先级:精确匹配优于整型提升,提升优于标准转换,转换优于用户定义转换。
- `const` 成员函数中 `this` 指向 `const` 对象,不能修改普通成员;`mutable` 成员是例外,允许在 `const` 函数里修改,适合缓存与计数等内部细节。
- `[=]` 把用到的变量拷贝进闭包,安全但可能拷贝大对象;`[&]` 存引用,零拷贝但依赖被引用对象存活。按引用捕获的 `lambda` 若被存到作用域之外再调用,引用悬空,是未定义行为。

实验:学生成绩报表生成器

本实验用一个完整的报表程序串联本章全部知识点:重载的 print、带默认参数的打印函数、引用参数读入数据、const 成员函数、函数对象、lambda、std::function 回调。代码分三部分,逐块阅读,每个覆盖点已在注释中标明。程序从标准输入读入分数(负数结束),输出人数、平均分、高于平均的人数与及格率。

实验 4•第一部分:声明、重载与引用参数(覆盖:重载、默认参数、引用参数、const 成员函数)

```
#include <iostream>
#include <string>
#include <vector>
#include <functional>
#include <algorithm>
#include <numeric>

struct Student {                                // 覆盖:const 成员函数
    std::string name;
    double score;
    const std::string& getName() const { return name; }
    double getScore() const { return score; }
};

// 覆盖:函数重载,同名按参数类型区分
void print(double v) { std::cout << "分数:" << v << "\n"; }
void print(const std::string& s) { std::cout << "姓名:" << s << "\n"; }

// 覆盖:默认参数,靠右,声明处给出
void print(const Student& st, int rank = 0) {
    std::cout << "第" << rank << "名:" << st.getName()
               << " " << st.getScore() << "\n";
}

// 覆盖:引用参数,零拷贝写出结果
bool readScores(std::vector<double>& v) {
    double x;
    while (std::cin >> x && x >= 0.0) v.push_back(x);    // 负数结束输入
    return !v.empty();
}
```

实验 4•第二部分:函数对象与 lambda(覆盖:函数对象、lambda、std::function)

```
// 覆盖:函数对象,operator() 携带状态(及格线)
struct ScoreFilter {
    double low;
    bool operator()(double s) const { return s >= low; }
};

// 覆盖:std::function 做回调参数,函数指针/lambda/函数对象皆可传入
void forEach(const std::vector<Student>& sts,
             const std::function<void(const Student&)>& cb) {
    for (const auto& s : sts) cb(s);
}

// 覆盖:默认参数 + lambda 按引用捕获,延迟到调用时执行
double passRate(const std::vector<double>& scores, double line = 60.0) {
    auto pass = [&](double s) { return s >= line; };
    int n = (int)std::count_if(scores.begin(), scores.end(), pass);
    return n * 100.0 / scores.size();
}
```

实验 4•第三部分:main 调度(覆盖:引用、lambda、函数对象、std::function、重载综合)

```
int main() {
    std::vector<double> scores;
    if (!readScores(scores)) return 0;           // 引用参数传出数据

    double avg = std::accumulate(scores.begin(), scores.end(), 0.0)
                / scores.size();

    // 覆盖:lambda 排序比较器,降序
    std::vector<double> sorted = scores;
    std::sort(sorted.begin(), sorted.end(),
              [](double a, double b) { return a > b; });

    // 覆盖:函数对象参与统计
    ScoreFilter aboveAvg{avg};
    int cnt = (int)std::count_if(sorted.begin(), sorted.end(), aboveAvg);

    std::cout << "共" << scores.size() << "人,平均" << avg
               << ",高于平均" << cnt << "人,及格率"
               << passRate(scores) << "%\n";

    // 覆盖:std::function 回调 + 带默认参数的重载
    std::vector<Student> cls{{"张三", 85.0}, {"李四", 59.5}, {"王五", 73.0}};
    forEach(cls, [](const Student& s) { print(s, 3); });
    print(cls[0].getScore());                    // 重载的 print(double)
    return 0;
}
```

实验覆盖点核对:重载(print 三个版本)、默认参数(passRate 与 print 的 rank)、引用参数(readScores 与 const 引用形参)、const 成员函数(getName/getScore)、函数对象(ScoreFilter)、lambda(排序与统计)、std::function(forEach 回调)、函数指针与名字修饰的概念在 §4.1 与 §4.5 已讲解,可自行把 forEach 的回调换成普通函数名验证互操作性。全部编译通过后,试着给 ScoreFilter 增加上界字段,或把 print 换成输出到文件的版本。

工程建议 1:接口参数先写 const 引用

新写一个函数时,参数一律从 const 引用起步,编译器报"需要修改"再放开成引用,只有确实允许为空才用指针。这样既避免大对象拷贝,又把只读契约写进类型里,团队协作时接口意图一目了然。

工程建议 2:回调与算法优先 lambda

标准库算法的比较器、谓词,线程与事件回调,一律用 lambda 表达;需要复用或状态复杂再升级为函数对象;只有需要动态替换实现(插件、测试替身)时才用 `std::function`。三个层次各司其职,代码可读性最好。

本章小结

C++ 函数在 C 之上新增了引用参数、默认参数与重载,并衍生出成员函数、函数对象、lambda、`std::function` 一整套可调用体系。传参三原则(小对象传值、大对象传 const 引用、可空传指针)、默认参数靠右、重载不按返回值区分、lambda 按值捕获安全按引用捕获易悬空——这五条是本章必须内化的结论。下一章进入 C 程序员最陌生的领域:内存管理。